SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 35%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

I _P .Lakshmi Gayathri Navaneetha __ (192321169) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

P.Gayathri
Signature of the student:

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

Answer 1: AI Drones for Emergency Medicine Delivery (Greedy Best-First Search vs A*)

1.1 Problem formulation

- **State:** drone position (grid cell / waypoint) plus current battery level.
- **Actions:** fly to a neighbouring waypoint.
- **Step cost g(n):** actual cost of a leg = distance × weather/terrain factor (wind, rain, flooded zones). It changes over time.
- **Heuristic h(n):** straight-line distance to the hospital/relief camp (optimistic because it ignores detours and weather).
- **Goal:** reach the delivery point with minimum delay and risk.

1.2 Behaviour of Greedy Best-First Search

Greedy search expands the node with the lowest **f(n) = h(n)**, i.e. it always flies toward the waypoint that looks closest to the destination in a straight line. It ignores the cost already spent.

In this scenario: the drone heads straight toward the target. If a flooded river or a blocked road lies on that line, the drone keeps being pulled toward it because the neighbouring nodes still look "close". It may fly into a dead end, then back out, and finally take a long detour.

Strengths under uncertainty	Weaknesses under uncertainty
Very fast: few nodes expanded, low memory, so decisions are made in milliseconds.	Not optimal: ignores weather and terrain cost g(n), so it may pick a cheap-looking but costly or dangerous route.
Cheap to re-run whenever a new blocked-path update arrives.	Can be incomplete: without a visited list it can loop between "close-looking" nodes.
Works with only a rough heuristic and no full cost model.	Easily trapped by dead ends and blocked corridors that the straight-line heuristic cannot see.
Good as an emergency fallback when time is almost zero.	No safety awareness: the shortest-looking path may pass through storms or flooded zones.

1.3 Behaviour of A*

A* expands the node with the lowest **f(n) = g(n) + h(n)**, where g(n) is the real accumulated cost from the start (distance adjusted for weather and terrain) and h(n) is the optimistic estimate of the remaining cost.

- **Balancing cost and heuristic:** g(n) keeps the search honest, because a path that has already been expensive (storm zone, long detour) is penalised. h(n) pulls the search toward the goal so that it does not explore uniformly in every direction.
- **When a road is blocked:** the node is removed (or given cost = ∞) and A* automatically falls back to the next best f-value on its open list, so a good alternative is already waiting.
- **Optimality:** A* returns the cheapest path if h(n) is admissible (never overestimates). Straight-line distance at best-case speed is admissible. If h(n) is also consistent, A* does not need to re-open closed nodes.
- **Risk handling:** risk can be built into the edge cost, for example cost = distance × (1 + weather penalty + flood penalty), so safe routes are preferred automatically.

1.4 Impact of heuristic accuracy

Heuristic quality	Greedy Best-First	A*
Accurate (close to true cost)	Goes almost directly to the goal and finds a good path quickly.	Expands very few nodes and stays optimal; the best case.
Underestimate (admissible but weak, e.g. h = 0)	Behaves almost randomly, with no guidance.	Still optimal but slow, since it degrades toward Dijkstra and expands many nodes.
Overestimate / misleading (e.g. flood forces a detour)	Badly misled: dead ends, long detours, unsafe paths. Very sensitive.	Faster but may lose optimality (non-admissible). g(n) still limits the damage.

Conclusion: Greedy depends completely on heuristic quality, whereas A* is protected by g(n) and degrades gracefully. In flood conditions the straight-line heuristic is often poor, so this protection matters.

1.5 Effect of dynamic changes (blocked paths)

- A plan computed earlier becomes **invalid** when a path is blocked, so the drone must **replan** (re-search) repeatedly.
- **Greedy:** replanning is cheap, but the new path may again lead into another blocked region or oscillate, and quality is unpredictable.

- **A*:** replanning from scratch each time is costlier (more nodes, more memory), but the result is reliable and optimal for the latest known map.
- **Improvement:** use incremental variants (**D* Lite, LPA***, Anytime Repairing A*) that reuse earlier search results and update only the affected part of the graph. Replan only when the path ahead is actually affected, and keep a short look-ahead horizon.
- Edge costs that change with weather mean h(n) must be recomputed using the best-case cost so that it stays admissible.

1.6 Recommendation

Recommended algorithm: A* (in its incremental/weighted form, such as D* Lite or Weighted A*), with risk-aware edge costs.

Criterion	Justification
Real-time constraint	Slightly slower than Greedy, but modern A* with a good heuristic is fast on waypoint graphs. Weighted A* ($f = g + w \cdot h$, $w > 1$) or D* Lite trades a small amount of optimality for speed when time is critical.
Optimality	Gives the minimum-cost (minimum delay) route, which matters because delay can cost lives and battery.
Safety	Weather, flood and obstacle risks go into g(n), so dangerous routes are avoided automatically. Greedy has no such mechanism.
Dynamic environment	D* Lite repairs the plan when roads are blocked without searching from scratch.

Fallback: Greedy search may be used only as an emergency mode when computation time is extremely short, with A* refining the route as soon as time permits.

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

Answer 2: Smart City Traffic Signal Optimization (Hill Climbing, Simulated Annealing, GA)

2.1 Formulation as an optimization problem

- **State (solution):** a vector of timing parameters for all N intersections, for example S = {cycle length, green split, offset} for each junction.

- **Neighbour / move:** change one parameter at one (or a few) intersections, for example +/- 2 seconds of green time or a shift of the offset.
- **Objective function (to minimise):** J(S) = w1·TotalWaitingTime + w2·FuelConsumption + w3·CongestionIndex, where the weights reflect city priorities.
- **Constraints:** minimum and maximum green time, pedestrian crossing time, safe clearance (amber/all-red), emergency vehicle priority.
- **Evaluation:** J(S) is computed from live detector data or a traffic simulator, which is expensive.

Why traditional (uninformed / systematic) search is inefficient

- **Huge search space:** with k timing options per intersection, there are k^N states. For k = 10 and N = 300 that is 10^300, far beyond BFS, DFS or A*.
- **No clear goal state:** the target is "as good as possible", not a specific configuration, so goal-test-based search does not apply.
- **No admissible heuristic:** the true remaining cost is unknown, and intersections interact (a change at one junction affects its neighbours).
- **Dynamic environment:** the best solution changes as traffic changes, so any slow systematic search is outdated before it finishes.
- **Memory:** storing frontier/explored sets for such a space is impossible. The path to the solution is irrelevant, so **local search** is the right family.

2.2 Hill Climbing in this scenario

1. Start from the current timing plan (or a random or heuristic plan).
2. Generate neighbours by small timing adjustments.
3. Evaluate J for each neighbour and move to the best one if it improves on the current J (steepest-ascent), or the first improving one (simple hill climbing).
4. Repeat until no neighbour is better, then stop.

Advantages: very low memory, fast, easy to implement and can run continuously, so it suits real-time fine-tuning. **Limitations:** greedy and purely local, with no way to go downhill, and the final result depends heavily on the start point.

2.3 Local maxima, plateaus and ridges (real-world meaning)

Problem	Meaning in the algorithm	Real-world traffic interpretation
Local optimum	A state better than all its neighbours but worse than the global best. The search stops.	A main corridor is tuned so well that every single change makes things worse, yet a different overall pattern (for example re-timing a whole district) would cut waiting time much more.
Plateau	A flat region where neighbours have equal J, so there is no direction to move.	In off-peak hours, changing a green phase by 2 seconds does not change delay measurably. The system wanders or stops with no signal about what is better.
Ridge	A narrow high region where each single move goes downhill, but a combined move goes uphill.	A "green wave" needs two or more adjacent signals to change their offsets together. Changing only one breaks

		coordination and increases delay, so hill climbing rejects the move.

2.4 Overcoming limitations: Simulated Annealing and Genetic Algorithms

Simulated Annealing (SA): accepts a worse neighbour with probability P = e^(-ΔJ/T), where T is a temperature that is lowered slowly (cooling schedule).

- At high T the system explores freely and can escape local optima and plateaus; at low T it behaves like hill climbing and settles into a good solution.
- Needs only one solution in memory and is easy to add on top of the existing hill-climbing code.
- Can be re-heated slightly when traffic patterns change significantly, which helps adaptability.

Genetic Algorithm (GA): maintains a **population** of timing plans, each encoded as a chromosome (the parameters of all intersections).

- **Fitness** = 1/J(S) (lower delay means higher fitness). **Selection** keeps good plans, **crossover** combines good sub-plans (for example the corridor timing of plan A with the district timing of plan B), and **mutation** introduces random changes.
- Crossover can change several intersections at once, so it crosses ridges that single-step hill climbing cannot.
- Searches many regions in parallel, which reduces the risk of being trapped. Multi-objective versions (NSGA-II) handle waiting time, fuel and congestion together.

2.5 Recommendation

Recommended approach: a hybrid of GA (global exploration) with Simulated Annealing / Hill Climbing (local refinement), applied in a rolling, district-wise manner.

Criterion	Justification
Scalability	The city is divided into zones; each zone runs GA/SA on its own intersections in parallel, and the evaluation of chromosomes is parallelisable. Complexity grows roughly linearly with the number of zones, not exponentially with intersections.
Escaping local optima	GA crossover/mutation and SA's probabilistic acceptance overcome local maxima, plateaus and ridges that defeat plain hill climbing.
Adaptability	The previous best population/solution is used as a warm start when traffic changes; fast hill climbing makes small real-time corrections between GA runs.
Practicality	Anytime behaviour: the best solution found so far can be deployed at any time. Safety limits (minimum green, pedestrian time) are enforced as constraints or penalties.

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search

ii. Analyze the challenges of partial observability and dynamic environments

iii. Describe how the rover builds and updates its internal model

iv. Compare online search with uninformed and informed search strategies

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

Answer 3: Autonomous Mars Rover (Online Search Agent)

3.1 Why an online search agent is required

An **offline** agent computes a complete plan before acting and then executes it. This needs a known state space, known actions and a known transition model. An **online** agent **interleaves planning and acting**: it plans a step, executes it, observes the result, and replans.

- **Unknown environment:** the map does not exist in advance, so offline search has nothing to search over. The agent can find out what an action does only by trying it.
- **Exploration problem:** the agent must discover states while pursuing its goal (collecting samples).
- **Irreversible and risky actions:** driving into a crater may be fatal, so the agent should act carefully on the latest information.
- **Communication delay:** one-way signal time between Earth and Mars is several minutes up to about 20 minutes, so remote step-by-step control is impractical and the rover must decide on its own.
- **Limited resources:** battery, time and computing power do not allow exhaustive offline planning of every contingency.

3.2 Challenges: partial observability and dynamic environment

Challenge	Explanation (Mars rover)
Limited sensing range	Cameras/lidar see only nearby terrain; areas behind rocks or crater walls are unseen, so the rover sees only part of the real state.
Noisy and uncertain sensors	Dust, shadows and low contrast cause misclassification of hazards; wheel slip and drift make position estimates inaccurate (localisation error).
Belief-state uncertainty	The agent cannot be certain of the true state and must reason about a set (or probability distribution) of possible states.
Dynamic changes	Dust storms, shifting sand, changing light, temperature and battery state alter the safety and cost of paths over time.
Exploration vs. goal pursuit	Exploring the unknown helps find better routes but wastes energy; too little exploration risks a bad route or dead end.
Dead ends / irreversibility	Some moves cannot be undone (stuck in sand), so the safe-explorability assumption can be violated.
Computation limits	On-board processors are slow, so planning must be quick and incremental.

3.3 How the rover builds and updates its internal model

- **Map:** an occupancy-grid / cost-map storing for each cell: unknown, free, obstacle, or hazard probability, along with terrain slope and roughness.
- **Transition memory:** a table result(s, a) recording where action a took the agent from state s, as used in online DFS and LRTA*.
- **Pose estimate:** position and orientation fused from wheel odometry, IMU and visual landmarks (SLAM).

- **Heuristic table H(s):** learned cost-to-goal estimates, updated during exploration.

Sense-Model-Plan-Act loop:

1. **Sense:** capture camera/lidar data and detect hazards in view.
2. **Update model:** mark new cells in the map (for example Bayesian log-odds update), correct the position, and revise H(s) or costs.
3. **Plan:** run a quick search (for example D* Lite / LRTA*) on the current map to choose the next safe action toward the goal or the nearest unexplored frontier.
4. **Act:** execute one short step, then return to step 1.

```
LRTA* update:  H(s) = min over actions a of [ c(s,a,s') + H(s') ]
Then move to the neighbour s' that gives the minimum value.
```

3.4 Online search compared with uninformed and informed search

Strategy	Strengths	Limitations on Mars
Uninformed (BFS, DFS, UCS)	Simple; BFS/UCS are complete and optimal on a known graph. Online DFS can explore with backtracking.	Offline versions need the full map. No sense of direction, so exploration wastes energy. Backtracking needs reversible actions; BFS memory grows too quickly.
Informed offline (Greedy, A*)	Directed toward the goal, efficient and (for A*) optimal.	Needs a complete, correct map. Plan is invalid when the model changes. Greedy can enter dead ends.
Online informed (LRTA*, D* Lite, frontier exploration)	Works with partial map; acts after each observation; learns and improves H(s); replans incrementally; can be made risk-aware.	Not guaranteed optimal at first visit; may revisit states; completeness requires a safely explorable space.

3.5 Most suitable approach

Recommended: an online, informed, risk-aware search such as D* Lite / LRTA* combined with frontier-based exploration and a conservative safety layer.

- **Uncertainty:** the map is updated at every step and the plan is repaired incrementally, so the rover is never relying on a stale map.
- **Adaptability:** D* Lite re-uses previous search effort when new obstacles appear, which is cheap enough for on-board hardware. LRTA* improves its heuristic on each visit.
- **Safety:** hazard probability and slope are added to the cost, and unknown cells are treated cautiously (a safety margin around obstacles). The rover keeps an escape/return path to a safe state, and a low-level reactive layer stops the rover for any immediate hazard.
- **Efficiency:** the heuristic (distance to the sample site) keeps exploration purposeful, while frontier exploration chooses informative cells when the goal direction is blocked.
- **Resource aware:** short horizons and incremental updates fit limited computation and battery.

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation
ii. Explain how backtracking search works in this scenario
iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
iv. Analyze real-world challenges and justify the best strategy for scalability

Answer 4: University Exam Timetable Generation (Constraint Satisfaction Problem)

4.1 CSP formulation

A CSP is defined by **(X, D, C)**: a set of variables X, a domain D for each variable, and a set of constraints C. A solution is a complete assignment that satisfies every constraint.

Variables

For each course (exam) i = 1..n, define two variables:

- **T_i:** the time slot of exam i (for example Day 1 morning).
- **H_i:** the exam hall assigned to exam i.
- (Optionally F_i: the invigilator / faculty assigned to exam i.)

Domains

- D(T_i) = {1, 2, ..., D×S}, the set of available slots (D days × S sessions per day), reduced by faculty availability.
- D(H_i) = {Hall_1, ..., Hall_m}, the halls with enough capacity for the enrolment of course i.

Constraints

Constraint type	Formal expression	Explanation
No student clash (binary)	If courses i and j share a student: $T_i \neq T_j$	A student cannot sit two exams at the same time. Courses with common students are connected in a conflict graph.
Hall capacity / availability	Capacity(H_i) ≥ Enrolment(i); if $T_i = T_j$ then $H_i \neq H_j$ (or total seats ≤ capacity)	Each exam needs a big enough hall, and two exams cannot use the same hall in the same slot unless seating allows.
Precedence (binary)	If i must be before j: $T_i < T_j$	Some subjects (for example a core course) must be examined before others.
Faculty availability (unary + binary)	$T_i \in$ Avail(faculty_i); faculty cannot invigilate two exams in the same slot	Respects staff timetables and workload.
Special accommodations (unary)	$H_i \in$ AccessibleHalls; duration/extra time; separate hall or no back-to-back	Students needing extra time, a separate room or a ground-floor hall are guaranteed suitable arrangements.
Soft constraints (optional)	Minimise students with 2+ exams per day / consecutive exams	Preferences used to score solutions, not strict rules.

Small illustration: courses A, B, C, D with slots {1, 2, 3}. A and B share students and A must be before C. Constraints: T_A ≠ T_B, T_A < T_C, and C and D may not use the same hall in the same slot.

4.2 Backtracking search

Backtracking is depth-first search that assigns **one variable at a time** and checks the constraints after each assignment. If a constraint is violated, it undoes the last assignment (backtracks) and tries another value.

```
BACKTRACK(assignment):
 if assignment is complete: return assignment
 var = SELECT-UNASSIGNED-VARIABLE()     # MRV / degree heuristic
 for value in ORDER-DOMAIN-VALUES(var): # least-constraining-value
    if value consistent with assignment:
       add {var = value}; result = BACKTRACK(assignment)
       if result != failure: return result
       remove {var = value}
 return failure
```

Walk-through (illustration): assign T_A = 1; then T_B must differ, so try T_B = 1 (violates A-B clash, rejected), T_B = 2 (accepted). Next T_C must be after T_A, so T_C = 2 (accepted); suppose D then has no consistent hall in any slot. The algorithm backtracks to the most recent choice, tries T_C = 3, and continues until all variables are assigned or all options are exhausted.

Improvements within backtracking: MRV (choose the variable with the fewest remaining legal values, for example a large exam that fits few halls), degree heuristic (choose the course with the most conflicts) and least-constraining value (pick the slot that leaves the most options to neighbours). Without these, the worst-case complexity is O(d^n).

4.3 Constraint propagation and forward checking

Forward checking: after assigning a value to a variable, remove from the domains of its unassigned neighbours all values that conflict with it. If any domain becomes empty, backtrack at once.

- **Example:** after T_A = 1, remove slot 1 from D(T_B) (shared students) and remove slots ≤ 1 from D(T_C) (precedence). If D(T_C) later becomes empty, the failure is detected immediately, before wasting time on deeper levels.
- **Benefit:** early detection of dead ends prunes large parts of the search tree and avoids repeating the same failures.
- **Stronger propagation:** arc consistency (AC-3 / MAC) also propagates the effect of removed values to neighbours of neighbours, which detects more inconsistencies at a higher cost per step but usually far fewer backtracks overall.

4.4 Real-world challenges and best strategy for scalability

Challenge	Effect / handling
Growth in courses and students	Search space grows exponentially (d^n); use heuristics, propagation, decomposition (for example by department or exam week) and the conflict-graph structure.
Over-constrained problems	There may be no solution that satisfies everything; split constraints into hard (must hold) and soft (preferences) and relax or optimise the soft ones (Max-CSP / weighted CSP).
Changes (late enrolment, hall unavailable)	Re-running from scratch is expensive; use local repair (min-conflicts) starting from the existing timetable.
Fairness and quality	Spread exams for students, avoid long gaps and overload; handle via soft-constraint cost functions.
Special cases	Accessibility and extra time are modelled as unary constraints to guarantee compliance.

Recommended strategy: a **hybrid approach**. First, model the problem as a CSP and solve the hard constraints with **backtracking + MRV/degree/LCV heuristics + forward checking / MAC (with conflict-directed backjumping)**, exploiting the conflict-graph (graph-colouring) structure and decomposing the problem into independent sub-problems where possible. Second, improve soft constraints and handle updates using **local search (min-conflicts, simulated annealing)** on the feasible timetable. This combines the completeness and correctness of systematic CSP search with the speed and flexibility of local search, and scales to large universities.

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

ii. Analyze the computational challenges of large game trees

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

Answer 5: Strategic Game AI (Minimax and Alpha-Beta Pruning)

5.1 How Minimax models the problem

Minimax models a **two-player, zero-sum, adversarial** game. The AI is **MAX** (tries to maximise the evaluation) and the human is **MIN** (tries to minimise it, that is, to win).

- **State:** the full game position (units, positions, resources, buildings).
- **Actions:** legal moves/orders available to the player whose turn it is.

- **Transition:** the new state after the move.
- **Utility:** value at terminal states (win = +1 or +∞, loss = -1 or -∞, draw = 0) or an evaluation function at the depth limit.

```
MINIMAX(s) = UTILITY(s)                    if s is terminal / depth limit
           = max over a of MINIMAX(RESULT(s,a))   if it is MAX's turn
           = min over a of MINIMAX(RESULT(s,a))   if it is MIN's turn
```

The AI assumes the opponent plays the best reply, so the chosen move guarantees the best worst-case outcome. For a real-time game, time is discretised into decision steps (or short action bundles), and unseen information (fog of war) is handled by assuming or sampling hidden states.

5.2 Computational challenges of large game trees

- **Exponential growth:** the tree has about b^d nodes. With branching factor b = 30 and depth d = 10 this is 30^10 ≈ 5.9 × 10^14 states, which is impossible to search in milliseconds.
- **High branching factor** in RTS games, because many units can each choose from many actions at once.
- **Strict time limits:** the AI must answer in tens or hundreds of milliseconds per decision.
- **Costly evaluation:** each leaf must be scored, and for complex states this is expensive.
- **Horizon effect:** limited depth means the AI may miss a danger just beyond its search depth.
- **Imperfect information and real-time simultaneity** make the exact minimax model only approximate.

5.3 How Alpha-Beta pruning improves efficiency

Alpha-Beta pruning returns **exactly the same decision as Minimax** but skips branches that cannot affect the final choice. It keeps two bounds:

- **α (alpha):** the best (highest) value MAX can already guarantee along the current path.
- **β (beta):** the best (lowest) value MIN can already guarantee along the current path.
- **Pruning rule:** stop exploring a node as soon as **α ≥ β**.

Reasoning: suppose MAX already has a move guaranteeing 3 (α = 3). While examining another move, MIN finds a reply that gives MAX only 2. MIN will pick that reply (or even better for MIN), so this move is worth at most 2 to MAX. MAX already has 3 elsewhere, so the remaining replies of this branch cannot matter and are skipped (a **beta cut-off**). The symmetric case at a MAX node is an **alpha cut-off**.

Worked example (root = MAX, three MIN children with leaves [3, 12, 8], [2, 4, 6], [14, 5, 2]):

Step	Result
MIN node 1: leaves 3, 12, 8	Value = 3, so α at root = 3.

MIN node 2: first leaf 2	2 ≤ α (3), so **prune the leaves 4 and 6** (beta cut-off). The node value ≤ 2 and is not chosen.
MIN node 3: leaves 14, 5, 2	14 and 5 do not allow a cut-off; value = 2.
Root (MAX) = max(3, 2, 2)	**3.** Same answer as plain Minimax, with 2 leaves skipped.

Efficiency: worst case O(b^d) (no pruning, bad move ordering). **Best case O(b^(d/2))** with perfect move ordering, which means the AI can search **about twice as deep in the same time**. Average case with reasonable ordering is about O(b^(3d/4)). Good move ordering therefore matters: try captures/attacks and the best move from the previous iteration first, and use killer and history heuristics.

5.4 Evaluation function design

Because the tree cannot be searched to the end, a depth-limited search scores non-terminal states with a heuristic **linear evaluation function**:

```
Eval(s) = w1·Army + w2·Economy + w3·Territory + w4·Tech + w5·Position + w6·BaseSafety (+ ...)
        each feature = (AI value) - (opponent value), normalised
```

Factor	Why it is considered
Army strength (w1)	Sum of unit values (health × attack power × cost) for both sides. Material advantage strongly predicts winning battles.
Economy (w2)	Resource income rate, workers, stockpile. A stronger economy converts to future army strength.
Map control / territory (w3)	Control of resource points, chokepoints and vision. Gives long-term advantage and information.
Technology / upgrades (w4)	Unlocked units and upgrades give a future power advantage.
Positioning (w5)	Unit formation, high ground, distance to enemy, threat and exposure of units.
Base safety (w6)	Health of key buildings and nearby enemy threats. Losing the base means losing the game, so this has a large weight.

Properties and justification: the function must be fast to compute (called at every leaf), strongly correlated with the real chance of winning, and exact at terminal states (win = +∞, lose = -∞). The weights are tuned by self-play, genetic algorithms or learning (for example temporal-difference learning), and may change with the game phase (economy matters more early on, army and base safety later).

5.5 Recommended strategy (balancing optimality and real-time performance)

Recommended: iterative-deepening, depth-limited Alpha-Beta search with strong move ordering and a tuned evaluation function, running as an anytime algorithm within a fixed time budget.

- **Iterative deepening with a time limit:** search depth 1, 2, 3... and stop when the time budget ends; always play the best move from the last fully completed depth. This guarantees an answer in time and also improves move ordering for the next iteration.
- **Move ordering + transposition table:** examine the most promising moves first and cache already-evaluated positions to avoid repeated work (this approaches the best case O(b^(d/2))).

- **Selective search:** beam/forward pruning to cut obviously bad moves, plus quiescence search (extend the search in unstable situations such as ongoing fights) to reduce the horizon effect.
- **Abstraction for RTS:** group units into squads and use high-level actions to cut the branching factor.
- **Alternatives for very large branching or hidden information:** Monte Carlo Tree Search (MCTS) can be used or combined with the evaluation function.
- **Trade-off:** the result is not guaranteed to be globally optimal, but it is near-optimal within the available time, and strong enough to be a competitive opponent.

Summary of recommendations

No.	Scenario	Recommended technique
1	Drone delivery in floods	A* (weighted / D* Lite) with risk-aware costs; Greedy only as emergency fallback
2	Traffic signal optimization	Hybrid GA + Simulated Annealing / Hill Climbing, zone-wise and rolling
3	Mars rover	Online informed search (D* Lite / LRTA*) with frontier exploration and a safety layer
4	Exam timetabling	CSP: backtracking + MRV/LCV + forward checking/MAC, then local-search repair for soft constraints
5	RTS game AI	Iterative-deepening Alpha-Beta with move ordering, transposition table and tuned evaluation

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand